# Time Series Imputation — Metric Implementation Verification

## How We Established That the Implemented Metrics Compute What They Claim

Every result in this thesis is a number a metric produced. A metric that is implemented incorrectly does not announce itself: it returns a plausible value, the value enters a ranking, and the ranking is wrong in a way no downstream check can recover. This document records what we did to rule that out, for the twenty metrics the pipeline scores and the two it implements but does not score.

The code described here lives in the repository at `https://github.com/ftrinca/metrics_exploration`, and every path below is relative to its root: `src/metric_eval/core/metrics.py` holds one function per metric, `src/metric_eval/core/metric_config.py` declares the registry, and `src/metric_eval/core/scoring.py` decides what each metric is evaluated on. Section 1 lists what is scored, and the sections after it give the four things we did about it.

## 1 The registry

Table 1 is `metric_config.py` written out. Twenty metrics are scored, grouped into five categories. Each carries a direction, which tells the ranking code whether a smaller or a larger value is better, and an evaluation scope, which decides whether the metric sees the missing positions alone or the whole reconstructed series.

Table 1: The scored registry. *Scope* is *masked* when the metric is evaluated at the missing positions only and *full* when it needs the whole series.

| Metric | Name | Better | Scope | Where the number comes from |
|---|---|---|---|---|
| **Pointwise Distance** | | | | |
| MAE | Mean absolute error | lower | masked | scikit-learn, `mean_absolute_error` |
| RMSE | Root mean squared error | lower | masked | scikit-learn, `root_mean_squared_error` |
| MSE | Mean squared error | lower | masked | scikit-learn, `mean_squared_error` |
| MRE | Mean relative error | lower | masked | written from the formula; positions with a zero truth are dropped |
| sMAPE | Symmetric mean absolute percentage error | lower | masked | written from the formula |
| nRMSE | Normalised RMSE | lower | masked | the same, divided by std($y$) |
| ND | Normalised deviation | lower | masked | written from the formula |
| **Distributional Divergence** | | | | |
| WD | Wasserstein distance | lower | masked | SciPy, `wasserstein_distance` |
| JSD | Jensen–Shannon divergence | lower | masked | SciPy, `jensenshannon`, squared, on shared-range histograms |
| KLD | Kullback–Leibler divergence | lower | masked | SciPy, `entropy`, on the same histograms |
| **Temporal Structure** | | | | |
| ACF | Autocorrelation agreement | lower | full | statsmodels, `acf`, FFT |
| DTW | Dynamic time warping | lower | full | `dtaidistance`, C backend, Sakoe–Chiba band |
| sMAE | Spectral mean absolute error | lower | full | SciPy, `lombscargle` |
| **Statistical Agreement** | | | | |
| Pearson | Pearson correlation | higher | masked | SciPy, `pearsonr` |
| MI | Mutual information | higher | masked | scikit-learn, `mutual_info_score`, on shared-range bins |
| $R^2$ | Coefficient of determination | higher | masked | scikit-learn, `r2_score` |
| TOST | Two one-sided tests | lower | masked | pingouin, `tost`, paired |

| Metric | Name | Better | Scope | Where the number comes from |
|---|---|---|---|---|
| BA | Bland–Altman | lower | masked | written from the formula; returns a pair |
| CDT | Cohen's distance test | lower | masked | written from the formula, which is Cohen's $d$ |
| **Domain-specific** | | | | |
| PFC | Proportion falsely classified | higher | masked | written from the formula, adapted to continuous data |

Two further metrics, CRPS and NLL, are implemented and deliberately kept out of the registry. Section 6 says why.

## 2 Delegating to a reference implementation

The first and cheapest defence is delegation. Thirteen of the twenty take their arithmetic from a library that is itself the reference for that quantity, so the part we could have got wrong is the part we never wrote: MAE, RMSE, MSE, nRMSE and $R^2$ come from scikit-learn, WD and Pearson from SciPy, ACF from statsmodels, DTW from `dtaidistance`, TOST from pingouin, and JSD, KLD and MI from a SciPy or scikit-learn primitive applied to histograms we construct.

The remaining seven are written out from the formula, because no library ships them in the form the imputation literature uses: MRE, sMAPE, ND, BA, CDT, PFC and the spectral sMAE, whose periodogram comes from SciPy but whose normalisation and comparison are ours. These are the seven where an implementation error is possible at all, and they are what the test suite in Section 3 is aimed at.

Delegation moves the risk, and it leaves two things to check. A library function can compute a slightly different quantity than its name suggests, and it can be called with the wrong arguments. Section 3 addresses the first by pinning each metric against an identity it must satisfy, and Section 4 records the places where the quantity we compute is knowingly not the textbook one.

## 3 Algebraic identities

A metric that is wrong by a constant factor, a missing square root or a transposed argument still returns a number that looks reasonable. It cannot satisfy an identity that the correct metric satisfies by construction. `tests/test_metrics.py` is built on that: it constructs a standardised series of 400 points with periodicity and autocorrelation, applies a transformation whose effect on the metric is known in advance, and asserts the value. The suite is 26 cases and all of them pass.

The identities fall into four groups.

Table 2: What each test pins, and what a failure would mean.

| Test | Identity | What it rules out |
|---|---|---|
| **Redundancy: a metric that carries no information another one does not** | | |
| ND is a fixed multiple of MAE | $\mathrm{ND}/\mathrm{MAE} = 1/\overline{\lvert y\rvert}$, the same for every reconstruction of one series | That ND ranks differently from MAE. It cannot. |
| nRMSE is a fixed multiple of RMSE | $\mathrm{nRMSE}/\mathrm{RMSE} = 1/\mathrm{std}(y)$ | The same, for the pair we chose the normaliser of. |
| MSE is RMSE squared | $\mathrm{MSE} = \mathrm{RMSE}^2$ | A missing or spurious square root in either. |
| **Error weighting: how the two pointwise metrics differ** | | |
| RMSE equals MAE under a constant offset | every error is the same size, so the two coincide | A wrong exponent: only a squared penalty collapses onto the linear one here. |

| Test | Identity | What it rules out |
|---|---|---|
| RMSE over MAE with concentrated error | error on a fraction $p$ of positions gives $\text{RMSE}/\text{MAE} = 1/\sqrt{p}$, checked at $p = 0.02, 0.05, 0.10, 0.25$ | Any weighting other than quadratic. The ratio is exact to $10^{-12}$. |
| **Blind spots: what a metric provably cannot see** | | |
| Pearson under a positive-slope affine map | $\text{corr}(y, ay + b) = 1$ for every $a > 0$, checked at four $(a, b)$ pairs | That Pearson responds to a shift or a rescaling. It does not, by construction. |
| $R^2$ under an offset | $R^2 = 1 - \alpha^2$ for an offset of $\alpha$ standard deviations | That $R^2$ shares Pearson's blind spot. It does not. |
| WD, JSD and KLD under a permutation | all three stay at zero | That any of the three reads temporal order. None does. |
| BA and CDT under a permutation, and under a rescaling about the mean | both stay at zero | That either sees more than the difference of means. |
| ACF and DTW under a permutation | both are strictly positive | That the temporal pair shares the distributional blind spot. |
| **Sanity: direction and monotonicity** | | |
| A perfect reconstruction scores perfectly | MAE, RMSE and WD at 0; Pearson and $R^2$ at 1 | A sign error or a transposed argument. |
| Lower-is-better metrics rise with noise | MAE, RMSE, MSE, WD, DTW increase over four noise levels | A direction declared wrong in `METRIC_DIRECTION`. |
| Higher-is-better metrics fall with noise | Pearson, $R^2$, MI decrease over the same four | The same, for the other direction. |

The blind-spot group is the part worth reading twice. Those tests fix in place what a metric fails to measure, so that a later result showing two metrics disagreeing can be traced to a known property: a bug would have failed the suite. When the algorithm ranking found the distributional metrics ordering a permuted reconstruction as perfect, the test suite had already established that this is what those metrics do.

Three identities in the table are tests and findings at once, and we report them as such. ND and nRMSE are fixed multiples of MAE and RMSE within one series, and MSE is RMSE squared, so each of the three produces exactly the ranking its partner produces. Measured on the test series, $\text{ND}/\text{MAE} = 1.2028 = 1/\overline{|y|}$ and $\text{nRMSE}/\text{RMSE} = 1.0000 = 1/\text{std}(y)$, the second being one because the data is standardised. Three of the twenty metrics therefore carry no ordering information that another one does not already carry.

# 4 Deviations from the canonical definition

Nine implementations depart from the textbook formula, in the eight rows below. Each departure is a choice, and each is recorded here with the reason, because a reader comparing our numbers against published ones needs to know which quantity was computed.

Table 3: Where the implemented quantity is not the textbook one.

| Metric | Deviation | Reason |
|---|---|---|
| MRE | Positions where the truth is exactly zero are dropped, so the denominator is the count of non-zero positions rather than $n$. | The ratio is undefined at a zero truth. PyPOTS's `calc_mre` drops the same positions, so our values remain comparable with the papers that use it. |
| nRMSE | Divides by $\text{std}(y)$. | The four papers reporting an nRMSE use four different denominators, so there is no convention to follow. Two of the four are unusable here: the mean, which IterativeSVD divides by, is zero by construction on standardised data, and TRMF's is deferred to an appendix that is not in the published paper. The MissForest denominator is the one that survives standardisation. |

| Metric | Deviation | Reason |
|---|---|---|
| sMAPE | Reported where the survey records MAPE. | MAPE divides by the true value and therefore explodes on standardised data. sMAPE divides by the mean of $\lvert y\rvert$ and $\lvert\hat{y}\rvert$ instead, which bounds it in $[0, 200]$; a reconstruction of $-y$ scores exactly 200. |
| JSD | Squares the value `scipy` returns. | `jensenshannon` returns the Jensen–Shannon *distance*, which is the square root of the divergence. The literature reports the divergence. |
| JSD, KLD, MI | Both series are binned over their shared range, into $\max(10, \lfloor\sqrt{n}\rfloor)$ bins; 20 bins at $n = 400$. | A shared range is what makes one bin label mean the same value interval in both series, which a joint histogram requires. ImputeGAP bins each series over its own range instead, so the two conventions give different numbers. |
| DTW | A Sakoe–Chiba band of 10% of the series length, so 40 timesteps at $n = 400$. | An unconstrained warp can match arbitrarily distant timesteps, which is both quadratic in cost and generous to a badly misaligned reconstruction. Measured on a lagged copy of the test series: a lag of 20 is inside the band and banded and unconstrained DTW agree exactly at 8.259; a lag of 40 sits at the band edge and the band raises the distance from 7.307 to 8.712. A separate check fixes the local cost: two constant series 400 points long and one unit apart give exactly $20 = \sqrt{400}$, which is the squared local cost with a single square root at the end rather than a sum of absolute differences. |
| BA | Returns the pair (mean difference, limit of agreement), which the scoring layer reduces to \|mean difference\|. | Bland–Altman is a plot. Ranking needs one number, and the bias is the half that answers "do these two agree on average". The limit of agreement is kept in the reports and dropped from the ranking. |
| PFC | Applied to continuous data as the percentage of positions within a 10% relative tolerance. | MissForest defines PFC as the proportion of misclassified categorical entries, and no continuous analogue exists. The adaptation is ours and the threshold is arbitrary; published PFC values and ours are not comparable. |

# 5 Evaluation scope and aggregation

Two decisions in `src/metric_eval/core/scoring.py` change every number the pipeline produces, and neither is visible in the metric functions themselves.

## What each metric sees

Seventeen metrics are evaluated at the missing positions only, which is the question the experiments ask: how well was the gap filled. The three temporal metrics are evaluated on the whole reconstructed series, because an autocorrelation, a warping path and a periodogram are all defined over consecutive time, and taking the missing positions alone would close the gaps between them and change the series being measured.

The size of that effect is easy to state. On the test series the lag-1 autocorrelation is 0.864; on the 80 scattered positions a 20% MCAR mask leaves, with the gaps closed up, it is 0.509. A metric evaluated on the subset would be reading a different series, so `FULL_SERIES_METRICS` holds ACF, DTW and sMAE and the mask is not applied to them.

## How several series become one number

The datasets are multivariate, so every metric is evaluated per series and the per-series values are averaged. The alternative, pooling every masked residual across all series into one array and taking a single value, is what ImputeGAP does, and for any metric that is not linear in the residuals the two disagree. On five series of increasing scale, the mean of the per-series RMSE is 2.979 and the pooled RMSE is 3.317, a difference of 11%,

while the two conventions give the same MAE to six decimals because the mean is linear and a mean of means is the global mean.

Neither convention is wrong. Per-series averaging weights every series equally regardless of its scale, and pooling weights every point equally. The consequence is that our RMSE and any RMSE-derived value cannot be set beside an ImputeGAP-reported number without saying which convention produced it.

### Missing scores

A metric that does not apply to a reconstruction, or that raises, is recorded as `None` rather than as a default value. The ranking places `None` last and every aggregation drops it, so a failed metric cannot enter a mean as a zero and quietly improve an algorithm's standing.

## 6 The two metrics we implement and do not score

CRPS and NLL are probabilistic scores, and both collapse onto a metric already in the registry when the reconstruction is a point estimate. The collapse is exact, and we verified both numerically on the test series.

- CRPS evaluated on a one-member ensemble is the mean absolute error. Measured: CRPS = 0.28567 and MAE = 0.28567, equal to every digit the float carries.
- Gaussian NLL with $\sigma$ estimated once from the residuals is $\frac{1}{2}\log(2\pi\,\mathrm{RMSE}^2) + \frac{1}{2}$, which is monotone in RMSE and therefore produces RMSE's ranking on a different scale. Measured: 0.41249 against a predicted 0.41247. The two agree to five significant figures rather than exactly, because the code estimates $\sigma$ as the standard deviation of the residuals and the identity uses the root mean squared residual; the two differ when the residuals are not exactly centred on zero.

Since every algorithm in the pipeline returns a point estimate, both would add a column to every table and no information to any of them. `metric_config.py` therefore holds them in `PROBABILISTIC_METRICS` and outside `METRIC_LIST`, and `metric_applies` scores them only when the reconstruction carries a samples dimension. Both functions have a branch for that case, taking the mean and standard deviation per timestep from the posterior samples rather than one global $\sigma$. That branch has never run, because no algorithm we use exposes posterior samples, and both docstrings say so.

## 7 Limitations

### Three metrics are unreliable on standardised data

All series are standardised before imputation, so the truth is centred at zero and a substantial number of positions sit close to it. Any metric dividing by the true value inherits that.

MRE is the clearest case. On the test series 12 of the 400 positions have $|y| < 0.05$ and the smallest is $1.5 \times 10^{-4}$. The zero-exclusion mask does not catch them, because they are near zero rather than zero, and each contributes a ratio of a normal error to a vanishing denominator. The result is an MRE of 6.64 where the MAE of the same reconstruction is 0.29, a factor of 23 driven by twelve positions.

PFC is affected in the same way and degrades where MRE explodes: a near-zero denominator makes the relative error large, the position fails the tolerance test, and it contributes zero to the average instead of corrupting it. The same reconstruction scores 19.0%, which reads as a poor result and is partly an artefact of the denominators.

ND escapes the problem, because it divides by $\sum |y_i|$ rather than by each $y_i$ in turn, and a sum of absolute values over a zero-mean series is strictly positive.

### TOST tests only the means

TOST asks whether the mean of the reconstruction is equivalent to the mean of the truth within a margin, which we set to $0.1\,\mathrm{std}(y)$. Any distortion preserving the mean passes it however badly the reconstruction is damaged. The injector experiment makes the size of this visible, because there every distortion is solved to the same pointwise damage of $0.5\sigma$: across its 192 calibrated reconstructions TOST declares 89 statistically equivalent to the truth. Broken down, it passes 21 of the 24 rescalings, 18 each of the 24 reorderings, noise

injections and smoothings, 14 of the 24 lags, and none of the 24 constant offsets, quantisations or spike injections. It separates the distortions that move the mean from those that do not, and nothing else.

One edge case is worth recording. When the difference between the two series is exactly constant *and* equal to the margin, both one-sided tests degenerate and pingouin returns `NaN`, which the scoring layer would pass on as a number rather than as a missing value. We checked all 192 cached TOST values from the injector run and none of them is affected, so the case is reachable in principle and has not occurred.

## The binning metrics depend on a choice we made

JSD, KLD and MI approximate continuous distributions with histograms, and their values depend on the bin count. The $\sqrt{n}$ rule with a floor of ten is a common default and it is still a default: more bins lower MI by sparsifying the joint histogram, and the same reconstruction scored with a different rule gives a different number. For MI specifically, `sklearn.mutual_info_regression` estimates the quantity from $k$-nearest neighbours and avoids binning altogether, which would be the better instrument for continuous data and is not what we use.

## What the tests do not establish

The suite pins identities alone. A metric that satisfies every identity in Table 2 and is nonetheless scaled wrongly by a constant would pass, provided the constant divides out of the ratios being asserted. The delegation in Section 2 is what covers that for the thirteen library-backed metrics, and for the seven written out by hand it is covered only where an absolute value is asserted: the perfect-reconstruction test, the $1/\sqrt{p}$ ratio and the two redundancy constants.

The suite also runs on one synthetic series. It is a series with the properties the metrics are meant to respond to, periodicity and autocorrelation and a standardised scale, but it is one series, and no test here is evidence about behaviour on the six real datasets beyond what the identities guarantee everywhere.